

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Основы комбинаторики

Комбинаторика – это раздел математики посвящённый решению задач, связанных с выбором и расположением элементов множества в соответствии с заданными правилами. Подобные задачи чаще всего возникают при составлении различных словарей для подбора комбинаций ключевых слов в паролях или логинах.

В стандартной библиотеке python есть несколько модулей, которые облегчают работу с этими задачами. Основным модуль, который мы рассмотрим это [itertools](#)

В этом модуле объединены функции, задача которых состоит в обработке последовательностей. Код функций использует не списки, а итераторы, благодаря чему скорость и эффективность работы увеличивается в разы.

```
In [1]: import itertools

In [2]: dir(itertools)
Out[2]:
['_doc_',
 '_loader_',
 '_name_',
 '_package_',
 '_spec_',
 '_grouper',
 '_tee',
 '_tee_dataobject',
 'accumulate',
 'chain',
 'combinations',
 'combinations_with_replacement',
 'compress',
 'count',
 'cycle',
 'dropwhile',
 'filterfalse',
 'groupby',
 'islice',
 'pairwise',
 'permutations',
 'product',
 'repeat',
 'starmap',
 'takewhile',
 'tee',
 'zip_longest']
```

Напоминаю, что для импортирования модуля мы используем инструкцию `import`, а

для получения информации о содержимом – функцию `dir()`. Для получения информации о конкретном методе, можно использовать встроенную справку `help()`.

Комбинаторные итераторы :

- `product()`
- `permutations()`
- `combinations()`
- `combinations_with_replacement()`

Предположим, собирая информацию о каком-то пользователе, мы узнали подсказку к его почте – дата рождения. Теперь нам нужно сформировать словарь, в котором будут все возможные варианты даты его рождения. Попробуем сформировать все комбинации дат с 1 января 1960 года до 1 января 2022 года. Можно пойти следующим путём – мы знаем, что в месяце может быть от 1 до 31 дня, месяцев может быть 12, диапазон годов так же нам известен.

Попробуем передать эти диапазоны в разные методы:

Метод	Количество	Возвращает
<code>product</code>	1191016	все возможные комбинации
Этот метод вернёт нам все варианты расположения элементов с повторениями. То есть каждый элемент будет находиться во всех положениях, от (1,1,1) до (2022,2022,2022)		
<code>permutations</code>	1157520	комбинации с последовательными перестановками
Этот метод создаст кортежи, в которых элементы переставляются друг за другом, то есть от (1, 2, 3) до (2022, 2021, 2020).		
<code>combinations</code>	192920	уникальные комбинации
Метод <code>combinations</code> вернёт только уникальные сочетания переданных элементов.		
<code>combinations_with_replacement</code>	204156	комбинации с повторами и возвратами
Этот метод аналогичен предыдущему с тем лишь исключением, что добавляет повторяющиеся элементы, то есть, если в <code>combinations</code> первым будет кортеж (1, 2, 3), то в этом – (1, 1, 1).		

Подумайте, каким именно методом лучше воспользоваться, чтобы получить оптимальный набор вариантов.

Рассмотрим наглядные примеры использования функций модуля `itertools`:

`product()` – прямое (Декартово) произведение одного или нескольких итераторов – это комбинирование объектов, в котором каждый объект может участвовать в размещении несколько раз.

Из слова "кот", мы получаем все возможные комбинации из 3-х символов (количество символов выставляется с помощью `repeat`). В `repeat` можно указывать любое количество элементов на выходе, и чем больше значение `repeat`, тем больше комбинаций получится.

```
1 from itertools import product
2
3 launch = 'кот'
4 launch_vars = product(launch, repeat=3)
5 for i in launch_vars:
6     print(i)
```

Run: new-test x

```
('к', 'к', 'к')
('к', 'к', 'о')
('к', 'к', 'т')
('к', 'о', 'к')
('к', 'о', 'о')
('к', 'о', 'т')
('к', 'т', 'к')
('к', 'т', 'о')
('к', 'т', 'т')
```

При необходимости получить на выходе строку, воспользуемся `join()`:

```
1 from itertools import product
2
3 launch = 'кот'
4 launch_vars = product(launch, repeat=3)
5 for i in launch_vars:
6     res = ''.join(i)
7     print(res)
```

Run: new-test x

```
kkk
kko
kkt
kok
koo
kot
ktk
kto
ktt
okk
oko
okt
```

`permutations()` – перестановки и размещения элементов множества. В этом варианте элементы не повторяются, и соответственно количество комбинаций меньше.

```
1 from itertools import permutations
2
3 launch = 'кот'
4 launch_vars = permutations(launch)
5 for i in launch_vars:
6     print(i)
```

Run: new-test x

```
( 'к', 'о', 'т' )
( 'к', 'т', 'о' )
( 'о', 'к', 'т' )
( 'о', 'т', 'к' )
( 'т', 'к', 'о' )
( 'т', 'о', 'к' )
```

Process finished with exit code 0

Мы можем сократить код, исключив цикл, и объявление переменной, воспользовавшись распаковкой кортежа *

```
1 from itertools import permutations
2
3 print(*permutations('кот'), sep='\n')
```

Run: new-test x

```
( 'к', 'о', 'т' )
( 'к', 'т', 'о' )
( 'о', 'к', 'т' )
( 'о', 'т', 'к' )
( 'т', 'к', 'о' )
( 'т', 'о', 'к' )
```

Process finished with exit code 0

Количество элементов можно выставить с помощью `r`, но при этом максимальное количество элементов не может быть больше чем количество исходных символов.


```
1  ▶ from itertools import permutations
2
3  launch = 'кот'
4  launch_vars = permutations(launch, r=2)
5  for i in launch_vars:
6      print(i)
```

Run: new-test ×

```
▶ ↑ ('к', 'о')
■ ↓ ('к', 'т')
|| ⏏ ('о', 'к')
|| ⏏ ('о', 'т')
|| ⏏ ('т', 'к')
|| ⏏ ('т', 'о')
⚡ ⏏ Process finished with exit code 0
```

`combinations()` – уникальные комбинации из элементов множества. Поскольку перестановок и повторений в этом варианте нет, то количество вариантов на выходе будет минимальным.

```
1  ▶ from itertools import combinations
2
3  launch = 'кот'
4  launch_vars = combinations(launch, r=2)
5  for i in launch_vars:
6      print(i)
```

Run: new-test ×

```
▶ ↑ ('к', 'о')
■ ↓ ('к', 'т')
|| ⏏ ('о', 'т')
|| ⏏ Process finished with exit code 0
```

Если взять длину `r`, равную длине слова, но вариант и вовсе останется один.

```
1  ▶ from itertools import combinations
2
3  launch = 'кот'
4  launch_vars = combinations(launch, r=3)
5  for i in launch_vars:
6      print(i)
```

Run: new-test ×

```
▶ ↑ ('к', 'о', 'т')
■ ↓
|| ⏏ Process finished with exit code 0
```

`combinations_with_replacement()` – комбинации с замещением (повторами, возвратами). На выходе получаются кортежи в отсортированном порядке с повторяющимися элементами.

```

1  ▶ from itertools import combinations_with_replacement
2
3  launch = 'КОТ'
4  launch_vars = combinations_with_replacement(launch, r=2)
5  for i in launch_vars:
6      print(i)

```

Run: new-test ×

```

('К', 'К')
('К', 'О')
('К', 'Т')
('О', 'О')
('О', 'Т')
('Т', 'Т')

```

Значение `r` можно выставлять любое.

```

1  ▶ from itertools import combinations_with_replacement
2
3  launch = 'КОТ'
4  launch_vars = combinations_with_replacement(launch, r=4)
5  for i in launch_vars:
6      print(i)

```

Run: new-test ×

```

('К', 'К', 'К', 'К')
('К', 'К', 'К', 'О')
('К', 'К', 'К', 'Т')
('К', 'К', 'О', 'О')
('К', 'К', 'О', 'Т')
('К', 'К', 'Т', 'Т')
('К', 'О', 'О', 'О')
('К', 'О', 'О', 'Т')
('К', 'О', 'Т', 'Т')
('К', 'Т', 'Т', 'Т')
('О', 'О', 'О', 'О')
('О', 'О', 'О', 'Т')
('О', 'О', 'Т', 'Т')
('О', 'Т', 'Т', 'Т')
('Т', 'Т', 'Т', 'Т')

```

Process finished with exit code 0